

1. اعرّف يعني إيه N+1 Problem وإزاي تحلها.

المشكلة دي بتحصل لما تعمل **استعلام (Query)** يجيب بيانات رئيسية، وبعدها لكل عنصر تعمل استعلام ثاني لوحده.

المشكلة:

- بطء شديد في الأداء
- ضغط على الداتابيز
- scalability ضعيف

مثال بسيط

عندك:

- Posts
- وكل Post له User (Author)

الطريقة الغلط:

```
$posts = Post::all();  
  
foreach ($posts as $post) {  
  
    echo $post->user->name;  
  
}
```

اللي بيحصل فعليًا:

- Query واحدة تجيب الـ 1 posts
- وبعدين لكل post يعمل query يجيب الـ N user

لو عندك 100 post:

👉 هتعمل $query\ 101 = 100 + 1$ 🤖

وده اسمه **N+1 Problem**

Laravel: Eager Loading في الحل

نستخدم **with()** عشان نجيب العلاقات مرة واحدة

```
$posts = Post::with('user')->get();

foreach ($posts as $post) {

    echo $post->user->name;

}
```

بدل Query 101: بقينا نعمل 2 Query بس:

1. posts
2. users المرتبطين بكل posts / الحل = Eager Loading (**with()**)

Search :

- Eloquent ORM:

هو نظام في Laravel بيخليك تتعامل مع الداتا بيز باستخدام **Models** بدل **SQL** مباشرة.

يعني إيه ORM؟

ORM = Object Relational Mapping

يعنى بتحول الجداول في الداتا بيز إلى Objects في الكود.

بدل `SELECT * FROM users;`

في لارافل `$users = User::all();`

مميزات Eloquent:

- كود سهل ومرتب
- علاقات بين الجداول (Relationships)
- CRUD بسهولة
- يدعم Eager Loading (يحل N+1 problem)

- **Query Builder:**

هو طريقة ثانية للتعامل مع الداتا بيز باستخدام **SQL-like syntax** داخل **PHP** بدون **Models**.

- `$users = DB::table('users')->get();`
- `$users = DB::table('users')->where('age', '>', 18)->get();`
- `DB::table('users')->insert(['name' => 'Ali', 'email' => 'ali@test.com']);`

الفرق بين Eloquent و Query Builder

Eloquent ORM :

ابطأ شوية -يدعم العلاقات -سهل وأوضح -يستخدم Models

Query Builder :

أسرع شوية -لا يدعم العلاقات بشكل طبيعي -أقرب ل SQL -يستخدم DB مباشرة

- استخدم **Eloquent**: في أغلب المشروع (CRUD + علاقات)
- ⚡ استخدم **Query Builder**: في الاستعلامات المعقدة أو الثقيلة